SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.

4. Suggest optimizations to improve performance.

Code of Conduct:

I (S.SHASHI VARDHAN REDDY/192311155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
S.SHASHI VARDHAN
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1: HTTP Response Analysis

1. Identify any incorrect or invalid fields in the header

The HTTP response header is:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Incorrect field:

Content-Length: -150 → Invalid

- The Content-Length value cannot be negative.
- It must be **0 or a positive integer** representing the number of bytes in the response body.

Other fields:

- HTTP/1.1 200 OK → Valid
- Date → Valid format
- Content-Type: text/html → Valid
- Connection: keep-alive → Valid

2. Analyze the issue with Content-Length and explain its impact

Issue:

- Content-Length is set to **-150**, which is not allowed by the HTTP specification. **Impact:**
- The client (browser or application) cannot determine the correct size of the response body.
- Some browsers may reject the response.
- Some clients may close the connection or display an error.
- It can lead to incomplete page loading or communication failures between the client and server.

3. Determine whether the status code matches the response correctness

Status Code: 200 OK

- 200 OK means the request was processed successfully and the server is returning a valid response.
- However, because the response contains an **invalid Content-Length header**, the response is **not correctly formatted**.

Conclusion:

- The **status code (200 OK)** does **not fully match** the response correctness.
- If the server cannot send a valid response, it should correct the header before sending it. If an internal problem prevents a valid response, an error status such as 500 Internal Server Error would be more appropriate.

4. Suggest corrections for the identified errors

The server should replace the invalid Content-Length with the correct size of the response body. **Correct Example:**

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
```

Content-Length: 150
Connection: keep-alive

(Assuming the HTML content is 150 bytes.)

If the exact length is unknown, the server should use an appropriate mechanism (such as chunked transfer encoding in HTTP/1.1) instead of sending an invalid negative value

.

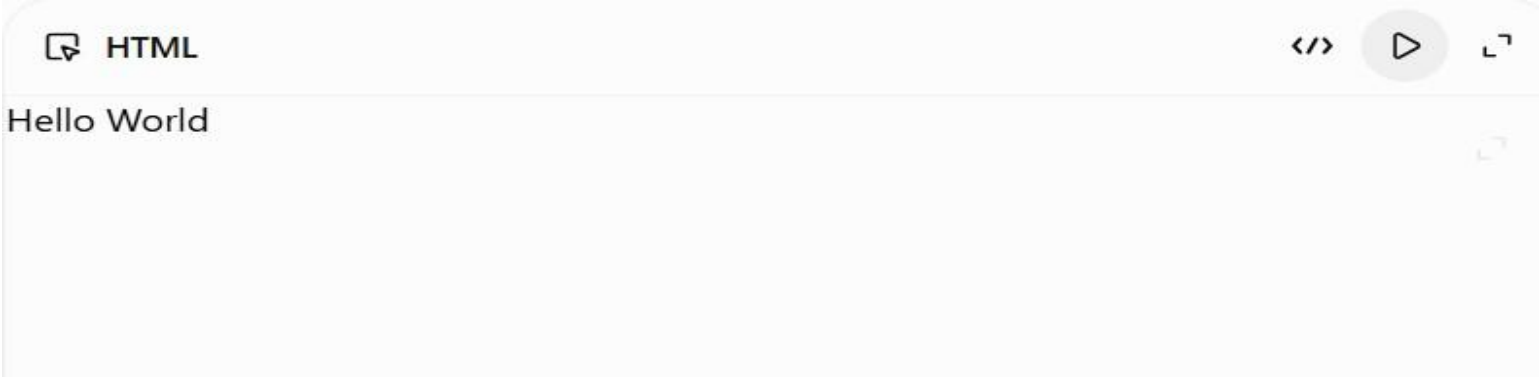
Outcome (Simple Language)

- **Incorrect field:** Content-Length: -150 is invalid because content length cannot be negative.
- **Impact:** The browser or client may reject the response or fail to load the webpage correctly.
- **Status code:** 200 OK indicates success, but the invalid header makes the response incorrect.
- **Correction:** Replace Content-Length: -150 with the actual positive size of the response body (e.g., Content-Length: 150) or use a valid transfer mechanism if the size is not known.

2. DOM Structure Analysis

Given DOM Structure

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>Hello World
</div>
</body>
</html>
```



1. Identify missing or improperly closed elements

Errors found:

- The `<p>` (paragraph) tag is **not closed**.
- The `<div>` tag is closed before the `<p>` tag is explicitly closed.
- Every opening HTML tag should have a matching closing tag for a valid HTML document.

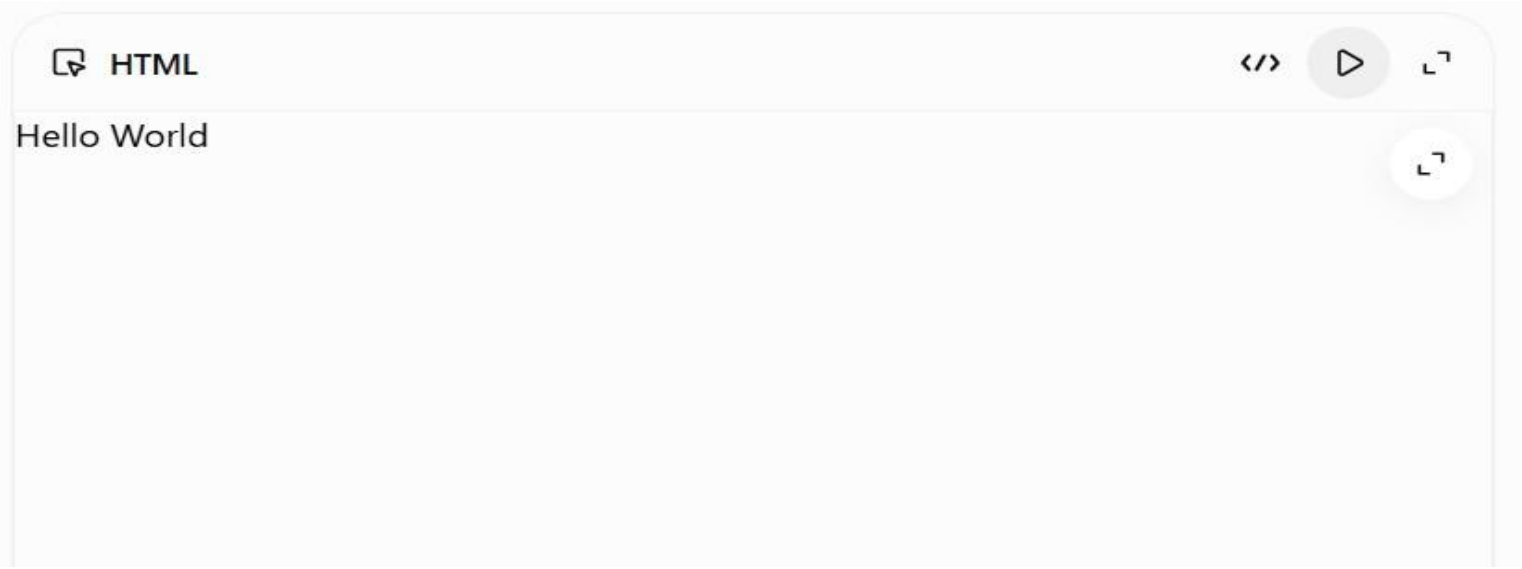
2. Analyze how the browser will interpret this structure

Modern web browsers automatically correct many HTML errors.
In this case, the browser will automatically insert the missing `</p>` tag before the closing

.

The browser interprets it as:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>Hello World</p>
</div>
</body>
</html>
```



So the page will usually display **"Hello World"** inside a paragraph within the `<div>` element.

3. Determine the impact on rendering and layout

Impact:

•Most modern browsers will display the page correctly because they automatically fix the missing closing tag.

•The layout will generally appear normal.

•However, invalid HTML can:

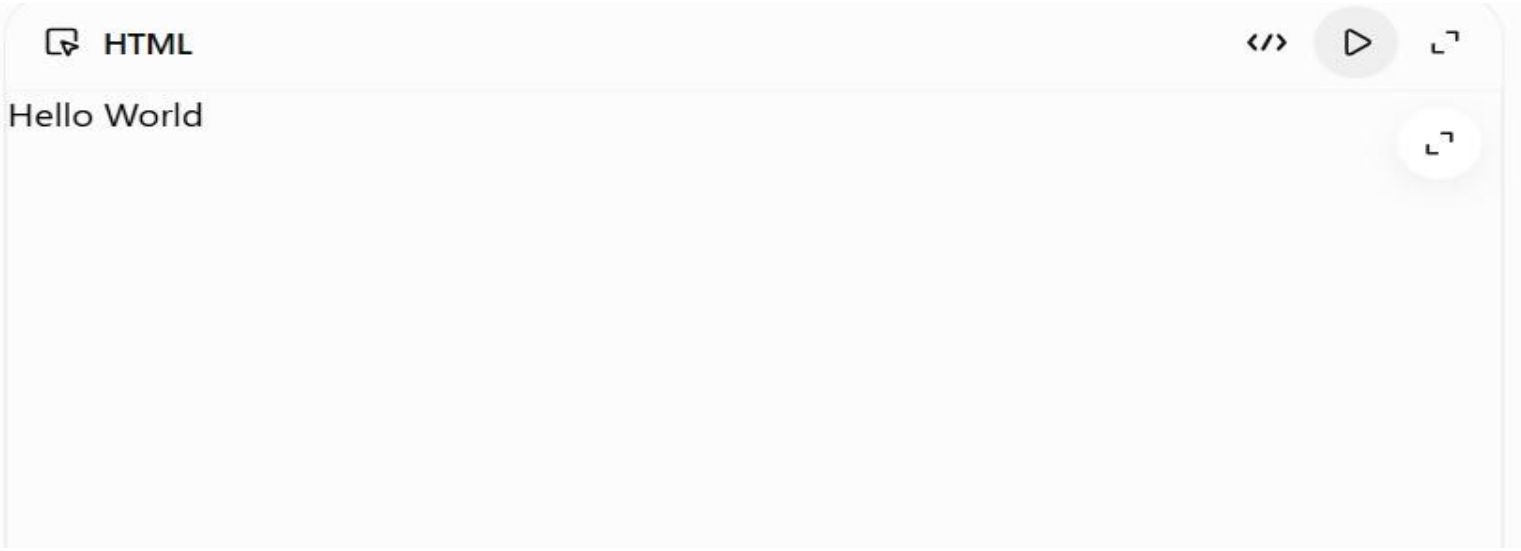
- Cause inconsistent behavior across different browsers.
- Make debugging more difficult.
- Create problems for screen readers and other accessibility tools.
- Lead to unexpected issues when using JavaScript to access or modify the DOM.

4. Suggest corrections to make the DOM valid

The <p> element should be properly closed before the tag.

Correct HTML

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>Hello World</p>
</div>
</body>
</html>
```



This version follows proper HTML syntax and creates a valid DOM structure.

Outcome (Simple Language)

- **Missing element:** The </p> closing tag is missing.
- **Browser behavior:** The browser automatically adds the missing </p> tag before .
- **Impact:** The page usually displays correctly, but invalid HTML can cause browser compatibility, accessibility, and JavaScript issues.
- **Correction:** Add the missing </p> tag before the closing to make the HTML valid.

3: GET vs POST in Login System

1. Differences in data transmission between GET and POST

GET	POST
Sends data in the URL	Sends data in the request body
Data is visible in the address bar	Data is hidden from the URL
Limited data size	Can send larger amounts of data
Can be bookmarked and cached	Not cached or bookmarked by default
Less secure	More secure than GET

2. **Which method is more suitable for login and why?**

POST is more suitable for login because:
•Username and password are sent in the **request body**, not the URL.
•Credentials are not stored in browser history or bookmarks.
•It provides better security, especially when used with **HTTPS**.

3. **Security risks of using GET for authentication**

•Username and password are visible in the URL.
•Credentials may be stored in browser history.
•Sensitive data can appear in server logs.
•URLs can be shared accidentally, exposing login details.
•Easier for attackers to intercept if HTTPS is not used.

4. **Best practices for secure login implementation**

•Use the **POST** method for login forms.
•Always use **HTTPS** to encrypt data during transmission.
•Hash and securely store passwords (e.g., bcrypt or Argon2).
•Validate input on both client and server.
•Use secure session management and cookies.
•Enable Multi-Factor Authentication (MFA) where possible.
•Protect against brute-force attacks with rate limiting or account lockout.
Conclusion:
POST + HTTPS is the recommended approach for login systems because it protects user credentials and provides a much higher level of security than **GET**.

4: HTML Code Inspection

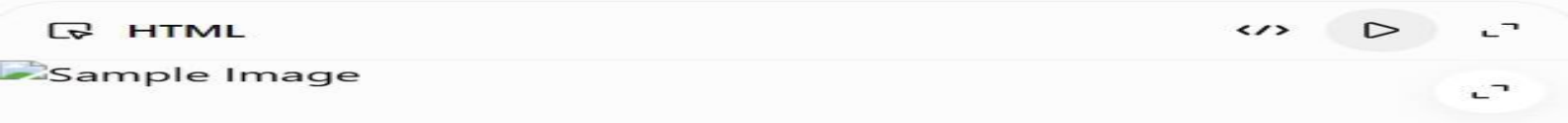
1. Identify missing closing tags

The HTML code has the following missing closing tags:
- </h1> for the heading.
- </p> for the paragraph.

2. Analyze issues with the <img> tag

- The <img> tag is missing the **alt** attribute, which is important for accessibility.
- Although <img> is a **void element** and does **not** require a closing tag, it should include descriptive alt text.

Example:
<img src="image.jpg" alt="Sample Image">



3. How browsers handle these errors

•Modern browsers automatically close unclosed tags.
•The page will usually display correctly despite the errors.
•Different browsers may interpret the HTML slightly differently, leading to inconsistent rendering.
•Invalid HTML can affect accessibility and maintainability.

4. Corrected HTML code

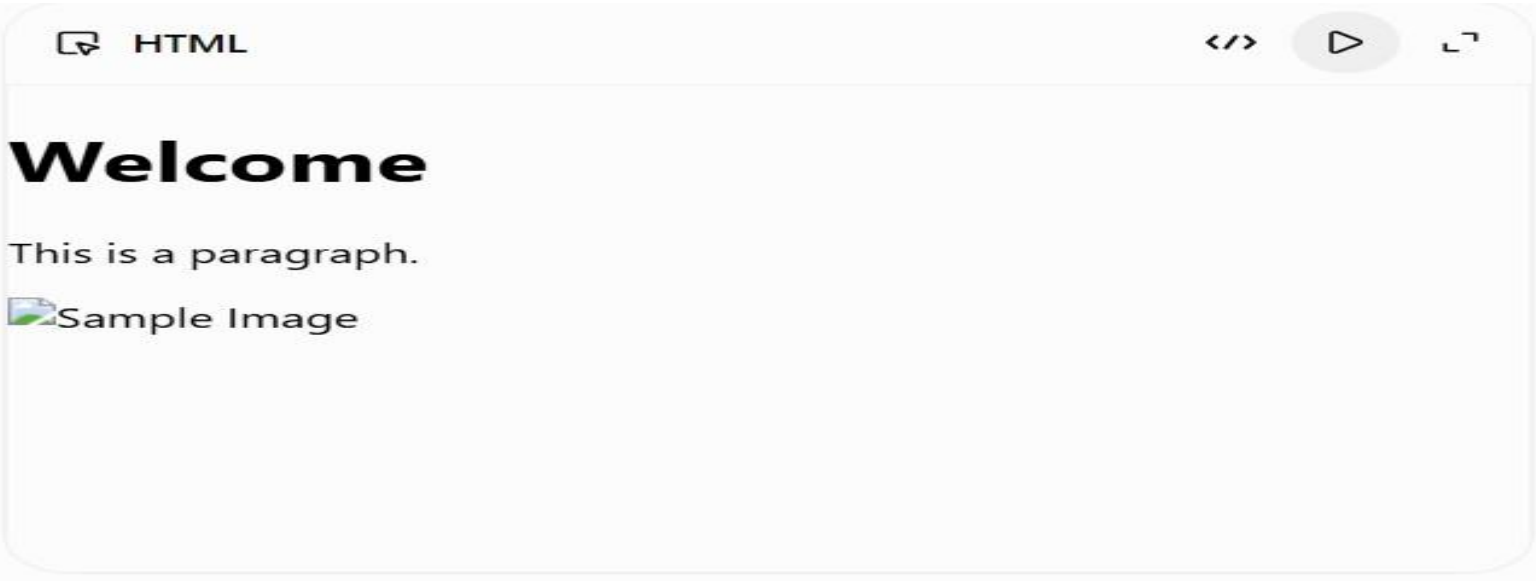
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>Sample</title>
</head>
<body>

    <h1>Welcome</h1>

    <p>This is a paragraph.</p>

    <img src="image.jpg" alt="Sample Image">

</body>
</html>
```



Result: The corrected code follows HTML5 standards, includes all required closing tags, and improves accessibility by adding the alt attribute to the image.

5: Browser-Server Communication Flow

1. Interpret each step in the communication flow

Step	Explanation
	The user types a website address (e.g.,

User enters a URL www.example.com) into the browser.	
DNS lookup occurs	DNS converts the domain name into the server's IP address.
HTTP request is sent to the web server.	The browser sends an HTTP/HTTPS request
Server processes request	The server processes the request, retrieves data, and generates a response.
Response is returned	The server sends an HTTP response (HTML, CSS, JavaScript, images, etc.) to the browser.
Browser renders page	The browser downloads resources, interprets the HTML/CSS/JavaScript, and displays the webpage to the user.

2. Identify where delays can occur

•Slow **DNS lookup**.
•Poor or slow internet connection.
•High network latency.
•Server processing delays.
•Large files (images, videos, CSS, JavaScript).
•Slow browser rendering or JavaScript execution.

3. Analyze the role of DNS

•DNS (**Domain Name System**) translates a domain name into its IP address.
•Without DNS, the browser cannot locate the web server.
•It acts like the **Internet's phonebook**, helping browsers find websites.

4. Suggest optimizations to improve performance

•Use **DNS caching** to reduce lookup time.
•Enable **browser caching**.
•Compress files using **Gzip** or **Brotli**.
•Optimize and compress images.
•Minify CSS and JavaScript files.
•Use a **Content Delivery Network (CDN)**.
•Upgrade to **HTTP/2** or **HTTP/3**.
•Reduce server response time and optimize database queries.

Conclusion:

A faster DNS lookup, efficient server processing, optimized resources, and caching techniques significantly improve website performance and reduce page loading time.